

Agentic AI 赋能科研

从想法到发表：一条 AI 研究流水线

相关工具：

ScholarEngineForAI — <https://github.com/pcwhy/ScholarEngineForAI>

ColabWatcher4aiAgents — <https://github.com/pcwhy/ColabWatcher4aiAgents>

主讲人：Dr. Yongxin Liu

数据科学助理教授 (Assistant Professor of Data Science) · Embry-Riddle Aeronautical University

Associate Editor, Journal of Big Data (《大数据杂志》副主编)

报告于休斯顿大学 Dr. Yunpeng (Jack) Zhang 研究组

2026 年 8 月

议程

1. 开场 — AI 改变的是研究的*执行方式*
2. 关键原则 — 上下文窗口、思维模式与分层流水线
3. 相关工具一 — Scholar Engine: 多阶段技术写作技能 (含第五阶段: 审稿修订)
4. 相关工具二 — Colab Watcher: 为本地 AI agent 提供 GPU 算力
5. 综合 — 一条完整的研究循环
6. 要点总结 & 问答

核心论点

AI 不只是改变怎么写研究——它改变的是研究如何被执行。

但一切始于一个值得回答的研究问题——在动用什么工具之前就要先验证
(Scholar Engine Phase I)：

好的 AI 增强研究 =
研究问题（什么值得研究——为什么）
+ 方法论（如何组织工作）
+ 基础设施（如何获得算力）
+ 人的判断（最终权威）。

第一部分

AI 增强科研的关键原则

上下文窗口问题

现代 AI 系统的上下文窗口是有限的：

- 一旦上下文过载，就开始发生**有损压缩**（lossy compression）。
- 结果是：AI 突然*“变得很笨，也更不主动”*。
- 整篇论文单会话完成的工作流会**无声地**退化——没有报错，只是输出变差。

“对于高质量学术研究与写作流程，现有 AI 系统的上下文窗口仍然不够长。”
— *ScholarEngineForAI README*

解决方案模式：分阶段 + memo 交接

- 把研究过程拆成四个阶段。
- 每个阶段结束时写一份 **memo**：做了什么、学到了什么、为下一阶段铺了什么路。
- 每个新阶段在新会话中开始——干净的上下文窗口。

Phase I → memo → Phase II → memo → Phase III → memo → Phase IV
(规划) (构建) (重构) (人工终审)

每个阶段都在满血状态下运行，从不被压缩。

工程师思维 vs. 科学家思维 —— 从第 0 天起认清你所问问题的性质

工程师: "这个系统可以这样做出来。"

→ 够一篇 EI 收录论文, 甚至可能 SCI。

科学家: "为什么别的系统做不出来? 是不是有没人研究过的隐藏规律? 背后的数学模型是什么? 这个系统在什么情况下会失败, 为什么?"

→ 这是专利和强 SCI 论文所需的思考层次。

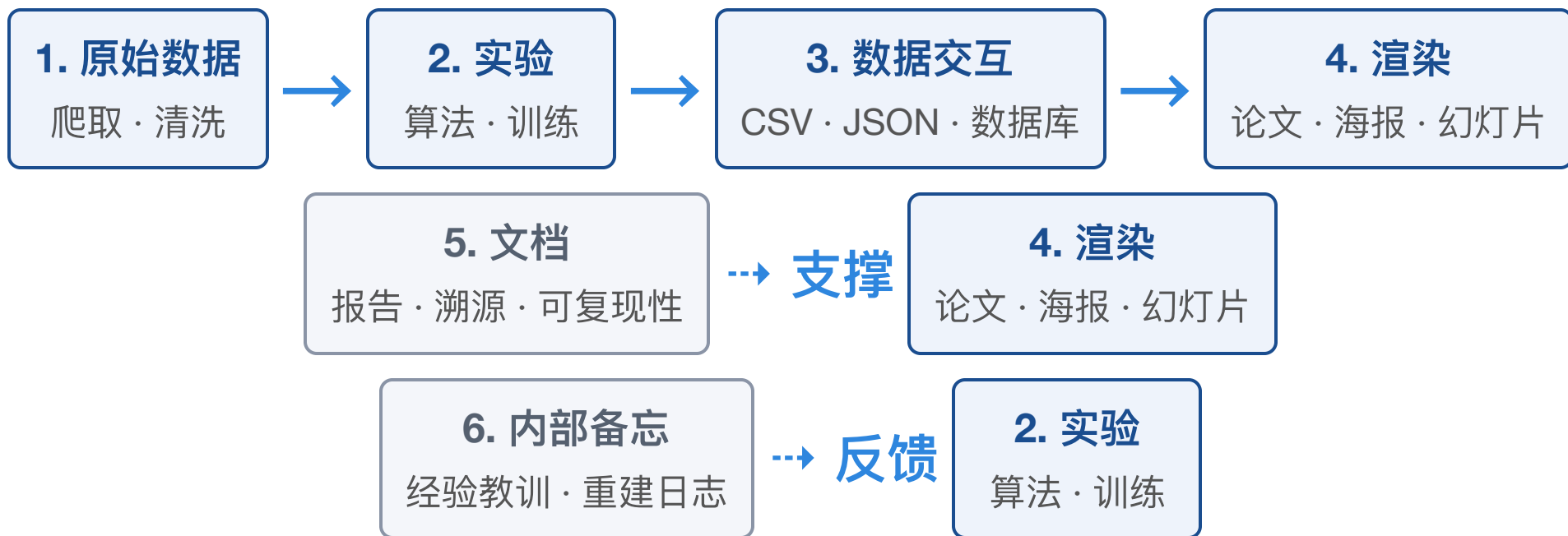
AI 必须基于事实、验证和推导一步步推理——不确定时直接说"我不知道"。

允许 AI 说"我不确定"...

允许说"我不知道"这个许可不是默认的——必须写进 prompt:

- 明确允许回答"我不知道"——否则模型会含糊其辞甚至编造;
- 要求每条未经验证的论断标注"未验证" (或删除);
- 要求每句话都能溯源: 来自数据、引自文献, 或明确承认是假设。

从系统工程视角看研究流水线



每一层都有自己的文件夹。不合并层、不纠缠逻辑。

为什么分层很重要

- 实验与 **LaTeX** 渲染分离——改格式永远不会重触发仿真。
- 数据交互层（CSV / JSON / 数据库）是所有渲染产物的**唯一事实来源**。
- 原始数据代码（爬虫）带时间戳保留——可复现、可审计。
- 两个能力必须始终成立：
 - 从零**全部重跑**并更新所有数据；
 - 更新数据 → 重新生成图表 → **重新生成 PDF**。

一个 prompt 永远不等于一篇论文

我们不能指望一个 prompt 一气呵成，而必须采用多阶段方法——正是因为单个会话根本无法产出高质量的论文：

- 上下文窗口溢出 → 有损压缩 → AI 变得*"笨得多、且不主动"
- 每个阶段以 **memo** 结束；每个新阶段在**新会话**中开始
- Phase I 规划 → Phase II 构建与草稿 → Phase III 重构 → Phase IV 人工打磨 → Phase V 审稿修订

经验法则：任何声称一次跑完整个流程的 prompt 或 skill，交给你的只会是课程作业，而不是出版物。拆分工作、预算上下文，让每个会话做好一件事。

第二部分

相关工具一 — Scholar Engine: 多阶段技术写作技能 (multi-stage technical writing skill)

这是我自己用 AI 写论文时，顺便生成的 skill。

Scholar Engine：仓库里有什么

一个技能包，安装进 AI agent：

文件	用途
README.md	四阶段方法——每个阶段该做什么
SKILL.md	操作协议：PI 式写作、流水线纪律——每个阶段怎么做、做到什么标准
section-protocols.md	分章节标准：摘要、引言、方法、相关工作、LaTeX 护栏
openai.yaml	仅 UI 元数据 — 技能在 agent 工具选择器中的显示方式

`openai.yaml` 里的 `prompt` 只是"一次处理一个产物"的界面元数据——"起草 / 改写 / 批判性审阅本小节"。它不是生成整篇论文的配方。

读一遍规范——这是对自己负责

- 请仔细阅读 ScholarEngine 中关于学术论文与做研究的具体规范——这是对自己负责的做法。
- 并非所有开源 skill 都可信：有人会在网上开源含有恶意指令的 **skill**。
- Skill 是给 agent 的"可执行指导"——安装前先审查，就像审查任何第三方依赖一样。

当前位置 — Phase I · 规划

1. Phase I · 规划 — 定调 · 验证 · 计划

2. Phase II · 构建 — 实现 · 草稿 · 文档

3. Phase III · 重构 — 修复 · 重构 · 故事

4. Phase IV · 打磨 — 人工终审

5. Phase V · 修订 — 审稿修订

本阶段产出：一个验证过的研究问题 + 一份书面计划文档。

Phase I.1 — 用强论文给 AI 定调

- 用 `arxiv-to-prompt` 或 `arxiv2md` 等工具，给 AI 喂一批高质量种子论文。
- 一箭双雕：
 - i. AI 理解你的研究语境；
 - ii. AI 学习你的写作风格。
- 跳过这一步，输出*"读起来像课程作业"。
- 这需要一个习惯：持续收集好论文。

Phase I.2 — 先验证研究价值，再制定计划

研究问题就在这里决定——在写任何一行代码或 LaTeX 之前。问题过不了这些测试，再好的方法论和算力也救不了。

让 AI 从五个角度对想法进行压力测试：

- **时效性** — 能否踩中当前趋势？
- **故事性** — 跨领域迁移、新颖发现、新范式？
- **问题价值** — 稀缺性、吸睛程度
- **跨学科性** — 从其他领域引入新思路
- **理论深度** — 更接近问题的根本原因

产出：一份书面计划文档。

参考文献预算：IEEE 会议论文 $\approx \geq 20$ 篇；期刊 ≈ 35 篇。

当前位置 — Phase II · 构建

1. Phase I · 规划 — 定调 · 验证 · 计划

2. Phase II · 构建 — 实现 · 草稿 · 文档

3. Phase III · 重构 — 修复 · 重构 · 故事

4. Phase IV · 打磨 — 人工终审

5. Phase V · 修订 — 审稿修订

本阶段产出：*实验、图表和达到会议水平的完整草稿——文档化且可复现。*

Phase II.1 — 实现、实验、草稿

- 尽量多探索组合与技术 → 获得**足够的数据**。
- 一个尝试/一个技术一个 **notebook**——绝不要一个大杂烩 notebook。巨型 notebook 是上下文和维护上的噩梦：重跑一次等于重跑全部。
- 产图的尝试单独放 **notebook**——重新生成图 N 不应重触发整个流水线。
- notebook 是主要产物；每个 cell 不超过 **300 行**；`.py` 文件 ≤ **400 行**（SKILL.md: 500）。
- **每个程序必须打印进度**——否则超时后，AI 会替你"简化"代码。
- **全部遵循匈牙利命名法**——否则代码库将无法维护。

Phase II.1 — 从一开始就分层

项目一开始就拆成：

raw data layer	— 爬取、清洗、导入（爬虫代码保留）
experiment layer	— 算法、训练、仿真（纯逻辑）
data interaction layer	— 原始输出存为 CSV/JSON（机器可读）
rendering layer	— 论文、海报、幻灯片（只从上一层生成）

AI 经常想抓额外数据——保留爬虫代码并标注开发时间。

必须人工检查： 确认爬虫代码和数据放在一起、待在原始数据层——爬虫代码正是那种会在“清理”时被悄悄删掉的一次性脚本。获取数据用到的每个 URL、命令、脚本都要保存，而不只是数据本身。

Phase II.1 — 隔离每个实验环境

- **不需要 GPU**：让 AI agent 为每个实验创建**虚拟环境（venv）**或 **Docker 容器**——绝对不要直接往系统里安装东西。
- **需要 GPU**：本地什么都不装——把任务交给后面要讲的 **Colab Watcher**。

Phase II.3 — 让 AI 帮忙探查模糊的问题与瓶颈

- 让 AI 检查系统在哪里失败：分类器（或系统）在哪些情况下出错？哪些边界条件会击穿它？
- 追问失败是否指向改进方向——是能修补的局限，还是设计层面的缺陷？
- 这正是 AI 的一大优势：扫一遍整体设计，帮你探查模糊的问题所在、定位瓶颈。
- 注意：AI 提出的改进方案不一定正确——必须用实验验证，不能盲从。

防御性写作：AI 如何写出高拒稿率的论文

防御性写作在文档里是必要的，但在论文里会毁掉它。写代码的 AI agent，尤其是 Codex，天生倾向含糊其辞、过度限定——在代码注释里没问题，在稿件里就是致命的。

- 审稿人会把防御性看成底气不足：它埋没你的主张，也给文章注水。
- 用配套技能 **anti-defensive-writing** 来清理：

<https://github.com/Kiterlin/anti-defensive-writing>

Phase II.2 — 摘要

- 摘要只写一段，不超过五分之一页。
- 背景/意义最多 4 行——其余写方法、亮点、结果。
- 只报你最得意的两个结果（或最有辨识度的两个发现）。
- 摘要要自洽独立：不出现内部简称或未解释的缩写。

Phase II.2 — 五段式引言

1. **背景** — 大图景，为什么这件事总体上重要
2. **缺口** — 前人做了什么、常见不足是什么
3. **动机** — 是什么启发了这篇论文
4. **贡献** — 逐条列出的贡献清单
5. **结构** — 论文如何组织

篇幅：**1 到 1.5 页**，不宜再长。

Phase II.2 — 什么才是真正的"贡献"——引言里审稿人和编辑会重点看的部分

- 贡献不是你做了哪些工作——没人关心这个。
- 贡献是：你发现了什么、回答了什么问题、突破在哪里。
- 最多 ≤ 4 条。

其余部分：逐节撰写（Write Other Parts Section-by-Section）

AI 可以一次成篇，但质量不好——要一节一节写，每节写完做质量控制。

- **方法（Methodology）**——必须人工检查：讲清楚系统各模块的输入输出；公式不要太多，优先照顾可读性。
- **实验与讨论（Experiments & Discussion）**——必须人工严审：防止 AI（特别是 Codex）防御性写作，导致论文可读性差、因而被拒；Discussion 必须分析本文方法在什么情况下性能不好、原因在哪——只汇报结果、不做深入分析，同样会被审稿人直接拒掉。
- **结论（Conclusion）**——只写两段：总结最重要的发现，以及依然存在的缺陷和未来可能的方向；每段不超过 1/5 页。

Phase II.2 — 专业的图表

- 图的质量反映你的论文图库——Nature/Science 级别的风格参考才值回票价。
- 表格用 `booktabs`；全文遵循 IEEE 风格。
- 每张图先生成 **PNG** 校样，插入前亲自检查。
- 所有图用一致的配色。
- 重跑实验 → 更新论文中的图——永远别忘了这个联动。

Phase II.2 — 画图的纪律

- 平时持续收集好图——你的图库就是风格参考和质量标杆。
- 给每张图定硬性规则：每图至少 **3 条曲线**、**字号不能太小**、**必须要有图例**。
- 画好第一张图后，让 AI 封装一个统一的画图函数——后续所有图继承同一风格。
- **GIS 可视化图**：先让 AI 分析样图、提取视觉特征（图层、配色、符号），再动手画。

Phase II.2 — 可选：AI 生成总览图

一张能概括整篇论文的高层示意图（Gemini Pro + NanoBanana——现在的 GPT 新模型也可以）：

- 输入：论文的**完整 LaTeX** + 你图库里的**风格参考图**；
- 平时**持续收集好图**效果更好——你的图库既是参考也是质量标准；
- **使用前必须人工检查**：检查每个视觉细节和每个文字标签——AI 生成图经常漏细节（少了翅膀的飞机、糊掉的文字）。

未经核验的 AI 图，绝不放入论文。

Phase II.4 — 强化一致性

- 一致性检查：
 - 无孤儿图/表——每个素材都要在正文中被引用；图/表的标题（caption）是最容易出现不一致的地方，务必仔细检查；
 - 不得未经人工审查就发明新概念；
 - 删除冗余和防御性表达；
 - 所有缩写首次出现时给出定义；
 - 避免长段落——每段最多 1/4 页；
 - 对防御性写作保持高度警惕——这是 AI 的底层习惯，任何 skill 都不可能完全消除。
- 第一轮技术报告就该达到普通会议论文的水平。

Phase II.5 — 文档化代码与素材

- 文档化代码（Markdown，必要时 Doxygen）。
- 对**每张图/表**：内容是什么、用途是什么、由哪段代码生成。
- 端到端验证两个能力：
 - i. 从零**全部重跑**并更新所有数据；
 - ii. 更新数据 → 重新生成图表 → **重新生成论文 PDF**。

当前位置 — Phase III · 重构

1. Phase I · 规划 — 定调 · 验证 · 计划

2. Phase II · 构建 — 实现 · 草稿 · 文档

3. Phase III · 重构 — 修复 · 重构 · 故事

4. Phase IV · 打磨 — 人工终审

5. Phase V · 修订 — 审稿修订

本阶段产出：*同样的发现，重组为一个承载价值的叙事。*

Phase III — 围绕价值与故事重构

1. **III.1** 修掉代码和写作里一眼可见的问题。
2. **III.2** 从标题到结论重新组织论文——让发现承载社会价值和/或讲出一个好故事。然后重写整篇论文。
3. **III.3** 再次删除冗余和防御性语言。

数据不变；变的是**框架和叙事**。只要有可能，**把稿子放下一周再回来看**——让问题自己浮现出来。

当前位置 — Phase IV · 打磨

1. Phase I · 规划 — 定调 · 验证 · 计划
2. Phase II · 构建 — 实现 · 草稿 · 文档
3. Phase III · 重构 — 修复 · 重构 · 故事
- 4. Phase IV · 打磨 — 人工终审**
5. Phase V · 修订 — 审稿修订

本阶段产出：一篇打磨好的稿件——清除假引用、清理幻觉、注入人味。

Phase IV — 人工终审

- 人仔细通读论文并打磨。
- 用 **Sourcely**、**Paperpal**、**Elicit** 等工具清除假引用。
- 删除重复措辞和幻觉内容。
- 注入真正的人味——这是 AI 无法替代的部分。

当前位置 — Phase V · 修订

1. Phase I · 规划 — 定调 · 验证 · 计划
2. Phase II · 构建 — 实现 · 草稿 · 文档
3. Phase III · 重构 — 修复 · 重构 · 故事
4. Phase IV · 打磨 — 人工终审
- 5. Phase V · 修订 — 审稿修订**

本阶段产出：干净的修订稿 + 逐条同步的回复信。

Phase V — 审稿修订：工作流程

审稿意见回来了——把这次修订当作新上下文中的新阶段，而不是在旧会话上打补丁：

- **(a) 按审稿人分文件接收**——把每位审稿人的意见分别放进单独的文件；在新会话中让 AI 先读你提交的稿子和审稿意见文件——读完只做确认 (**acknowledge**)，不做别的。
- **(b) 先计划，再动手**——让 AI 对审稿意见归类，产出分阶段修改计划（包括补实验）。你必须先审查一遍计划，批准后再执行；归类时一定要注意，不要把审稿人与问题的关联关系弄乱了。
- 然后按照计划逐阶段修改——见下一页。

Phase V — 逐阶段修改与内部回复信

- 按照计划逐阶段修改——不必每个修改阶段都开新会话；如果发现上下文窗口即将占满，再考虑开新会话。
- 每个修改阶段结束，一定要看 AI 写的总结：它是怎么改的；不合理的地方一定要当场指出，绝不要让问题传播到下一阶段。
- 从第一阶段起就维护一份内部临时 **Response Letter**——它随修订一起生长，任何要点都不会丢失；每个问题都要标注是哪个审稿人提的，因为归类时评论的次序可能被打乱。同时保留分阶段修订笔记——每一份都要有明确的 *Reviewer Comments Addressed*（已答复审稿意见）记录。

Phase V — 正式的回复信 (Formal Response Letter)

- 每位审稿人都要有单独的 **Response Letter**——AI 可以起草，但提交前必须人工检查：核对每一条回复与实际改动是否相符，绝不能悄悄丢给 AI 代写。
- 最终版回复信必须写清楚：具体做了哪些实验、怎么修改的、改动在本次提交稿的哪个位置。

Phase V — 正式回复信的格式与语气

- 用 **LaTeX** 写回复信——体现你的专业度；语言一定要容易读懂：Codex 很爱写只有自己看得懂的内部黑话，对审稿人来说十分刺眼，**绝对不能出现在回复信里**。
- 语气：专业、尊重、**非防御性**——先承认真实弱点，再解释修复；修复稿件和证据链，如果结果不再支持某个论断，就修改论断。

第三部分

相关工具二 — Colab Watcher: 为 AI agent 提供 GPU 算力

算力问题

- 写作和规划很便宜——训练和重实验不是。
- 本地 AI agent 能干，但受限于 **CPU**。
- Google Colab 提供免费/廉价 **GPU**——但会话是临时的、全靠手动操作。
- **ColabWatcher4aiAgents** 通过一个 **Drive** 文件夹给 agent 提供远程算力：无服务器、无 API 密钥、无 SSH——只是一个文件队列。

架构：把 Drive 当作消息总线



无服务器、无 API 密钥——notebook 运行 `watch_forever()`，agent 通过本地 **Drive** 镜像工作。

第一步准备：把 Drive 镜像到本地磁盘

本地 agent 从不调用 Google API——它通过你的本地 **Drive 镜像** 读写：

1. 安装 **Google Drive for Desktop**，并登录与 Colab 会话相同的账号；
2. 镜像 `Colab Agent/` 文件夹，让它变成普通本地文件夹；
3. 告诉 **AI agent** 镜像在本地哪里，并让它读 README / SKILL.md；
4. agent 把任务放进本地镜像 → Drive 同步 → Colab watcher 取走 → 结果流回 `done/`。

两侧看到的是同一个文件夹——文件系统就是共享总线。

Drive 目录：一个基于文件的任务队列

Colab Agent/	
control.md	← 操作开关 ("stop" / "run")
heartbeat.json	← 存活信号 + 遥测
jobs/	
inbox/	← 在此提交任务
running/	← 执行中（不要动）
done/	← 已完成工作区
failed/	← 失败与中止

无数据库。文件系统就是状态。

任务类型

类型	如何执行
单个 <code>.py</code>	直接执行
单个 <code>.ipynb</code>	通过 papermill 执行
打包文件夹	<code>job.json</code> 声明 <code>entrypoint</code>

```
{ "entrypoint": "main.py" }           // 或 "notebooks/run.ipynb"
```

安全规则：entrypoint 必须待在任务文件夹内（防路径穿越）。
只有一个可运行文件 → 自动识别；多个 → 必须提供 `job.json`。

Watcher 主循环

```
while True:
    write_heartbeat() # every 30 s
    if stop_requested(): sleep(); continue
    for job in list_pending_jobs():
        run_in_isolated_workspace() # timeout 30 min
```

- 每 **15 秒** 轮询；每 **30 秒** 心跳；任务超时 **30 分钟**
- 每个任务在自己的工作区 `jobs/running/<job_id>/` 中运行
- **stop** (`control.md`)：终止当前任务，但 watcher 保持存活
- 把不同任务分到不同的 **notebook/运行时**——一个 Colab 运行时死机，其余照常运行；单会话一卡，所有任务都会被拖死

遥测与控制

heartbeat.json 报告运行时健康状况:

```
{
  "timestamp_utc": "2026-08-10T12:00:00Z",
  "state": "watching",
  "control_stop_requested": false,
  "pending_jobs": 2,
  "runtime": {
    "memory": {"used_bytes": ..., "usage_percent": ...},
    "disk_free_bytes": ...,
    "gpu": {"devices": [{"name": "T4", "utilization_gpu_percent": 87}]}
  }
}
```

每个任务: execution.log + job_result.json → 状态 done / failed / stopped。

GPU 真相

- **GPU 版 Colab 运行时是必要条件，但不是充分条件。**
- `papermill` 不会自己启用 GPU——它只是把 notebook 跑在当前运行时里。
- **任务代码本身必须使用 GPU：**
 - PyTorch: `torch.cuda.is_available()` + 显式把张量搬到 CUDA；
 - TensorFlow: 常规的 GPU 感知配置。
- 启示：基础设施提供容量；实验决定使用。

测试用例: `ThreeLayerNN_Pass.ipynb`

来自 ERAU **DS615** 数据建模课程的三层 MLP，跑在 MNIST 上 ($784 \rightarrow 30 \rightarrow 20 \rightarrow 10$)。

研究思路：可视化**"pass patterns (通过模式)"** —

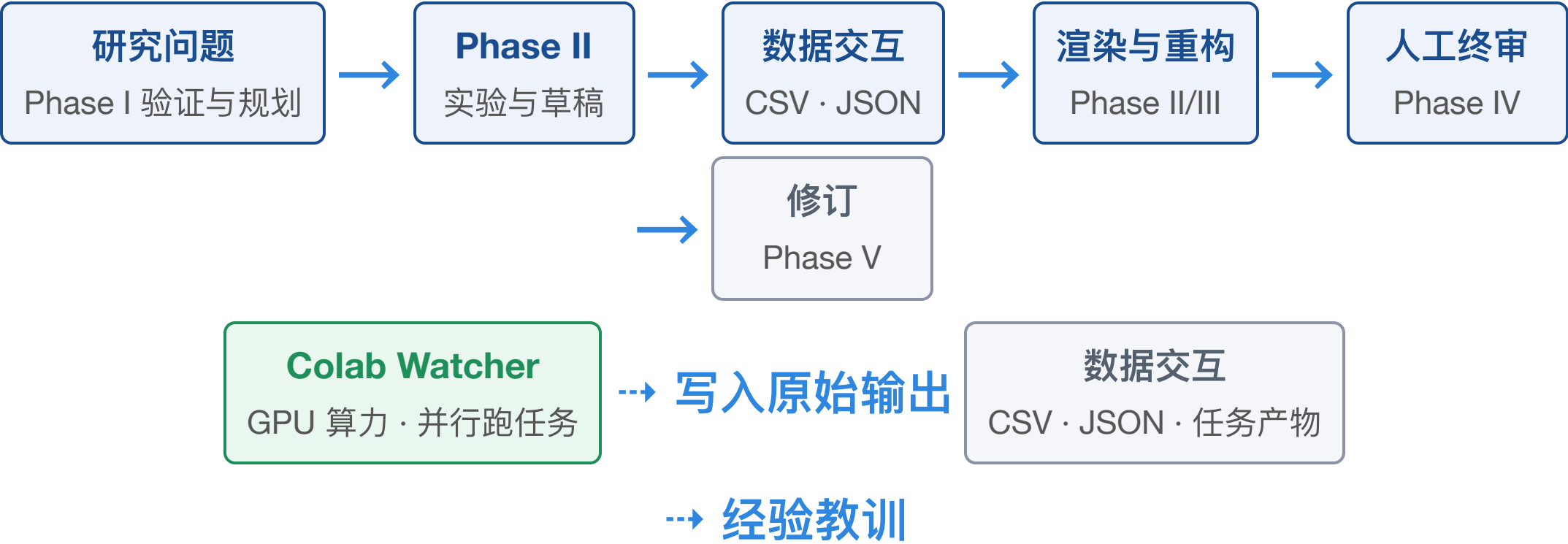
- ReLU 激活后的权重行，重塑成 **28×28** 图像；
- 跨层权重矩阵相乘得到**"融合模式"**——什么特征"通过"了网络。

正是那种本地 agent 可以排队、运行、再从 Drive 取回的 GPU 任务。

第四部分

一条完整的研究循环

完整循环



AI 执行循环；人始终是 PI — 规划把关与最终判断。

AI 时代准则

- 任何用 **AI** 辅助、用于发表的产出工程，都应视作一条带有分阶段质量控制的产品开发流水线（**product development pipeline**）——Phase I-V 就是它的质量门。
- 在 AI 时代，人人都要成为系统工程师：由你设计流水线、检查点，以及人在何处把关。

可迁移的经验

- 预算上下文 — 分阶段 + memo 交接，绝不用一个巨型会话。
- 层间解耦 — 数据、实验、渲染、文档、备忘。
- 让一切可观测 — 心跳、进度打印、执行日志。
- 把技能包装成机构知识 — 一份技能文件让任何会话都变成专家。离开去喝咖啡之前，先让 AI 把技能写好。
- 让人在关键时刻介入 — 想法验证、质量控制与最终把关。

要点总结

AI 增强研究是一种**流水线纪律**——而且从一个问题开始：

提出对的问题 · 结构化工作 · 预算上下文 · 租用算力 · 人始终是 PI。

仓库： <https://github.com/pcwhy/ScholarEngineForAI> ·

<https://github.com/pcwhy/ColabWatcher4aiAgents>

资源

用途	工具
种子论文 → prompt	arxiv-to-prompt , arxiv2md
notebook 执行	papermill
查证参考文献	Sourcely, Paperpal, Elicit
总览图	Gemini Pro + NanoBanana
技能打包	SKILL.md + openai.yaml (+ Codex/agents)

谢谢，欢迎提问！